EXPERIMENT 1

rpc.py Python

```python
import socket
import json
import inspect
from threading import Thread

SIZE = 1024

class RPCServer:
    def __init__(self, host="0.0.0.0", port=8080):
        self.host = host
        self.port = port
        self.address = (host, port)
        self.methods = {}

    def register_method(self, function):
        self.methods[function.__name__] = function

    def register_instance(self, instance):
        for name, func in inspect.getmembers(instance, predicate=inspect.ismethod):
            if not name.startswith("_"):
                self.methods[name] = func

    def handle(self, client, address):
        print(f"Client connected: {address}")

        while True:
            try:
                data = client.recv(SIZE).decode()
                if not data:
                    break
                function_name, args, kwargs = json.loads(data)
                print(f"Request: {function_name}{tuple(args)}")
                result = self.methods[function_name](*args, **kwargs)
                client.sendall(json.dumps(result).encode())

            except Exception as e:
                client.sendall(json.dumps(str(e)).encode())

        print(f"Client disconnected: {address}")
        client.close()

    def run(self):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.bind(self.address)
            s.listen()
            print(f"RPC Server running on {self.address}")
            while True:
                client, address = s.accept()
                Thread(target=self.handle, args=(client, address)).start()

class RPCClient:
    def __init__(self, host="localhost", port=8080):
        self.address = (host, port)
        self.sock = None
    def connect(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.connect(self.address)
    def disconnect(self):
        if self.sock:
            self.sock.close()
    def __getattr__(self, name):
        def execute(*args, **kwargs):
            request = json.dumps((name, args, kwargs))
            self.sock.sendall(request.encode())
            response = self.sock.recv(SIZE).decode()
            return json.loads(response)
        return execute
```

client.py Python

```python
from rpc import RPCClient

client = RPCClient("localhost", 8080)

client.connect()

print(client.add(5, 6))
print(client.sub(5, 6))

client.disconnect()
```

server.py Python

```python
from rpc import RPCServer

def add(a, b):
    return a + b

def sub(a, b):
    return a - b

server = RPCServer()
server.register_method(add)
server.register_method(sub)
server.run()
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh - expt_1

```
expt_1 git:(main) ✗ python3 client.py
11
-1
expt_1 git:(main) ✗
```

```
expt_1 git:(main) ✗ python3 server.py
RPC Server running on ('0.0.0.0', 8080)
Client connected: ('127.0.0.1', 56265)
Request: add(5, 6)
Request: sub(5, 6)
Client disconnected: ('127.0.0.1', 56265)
```

zsh expt_1
Python...